本科毕业论文

面向持久内存的文件系统设计与实现

基于 xv6 的 PMFS 文件子系统与可机器判定契约

专业名称：计算机科学与技术

学生姓名：（姓名）

指导教师：（指导教师）

所在学院：（学院）

完成日期：二〇二六年六月

（按学校正式封面模板替换本页）

摘　要

持久内存（Persistent Memory, PMEM）以字节寻址方式提供近似 DRAM 的访问延迟，同时具备掉电后数据保留能力，为操作系统存储子系统提出了不同于块设备日志文件系统的一致性模型与工程验收问题。教学操作系统 xv6 以 virtio 块设备与日志型文件系统作为根文件系统，适合讲授经典 UNIX 语义，但难以直接承载「可机器判定的崩溃与掉电语义」这一工程化目标。

本文按原题四要件，在 xv6-riscv 上实现并行于根文件系统的字节寻址 **PMFS 文件子系统**：映射 QEMU NVDIMM 仿真窗口；以 inode/dentry 页与双槽超级块重构去块设备布局；文件写入经 CDC/stream 切分后融合至 KV/run 核；以缓存行粒度 `pmem_flush`+`fence` 有序协议保障掉电安全。后端含 EMUL、MAP 与 PMEM2（`memory-backend-file`）。语义上写死崩溃可见点与 POSIX 差集：仅 `fsync`/提交后写可见。

核心贡献是教学 OS 上**可机器判定的崩溃与掉电契约套件**（含 `PERSIST`/`FILE`/`REMOUNT`/`FAULT`/`P`
实验分两套口径：架构面报告文件 API 相对原生 `fs+log` 的延迟中位数与相对吞吐；公平面以 A0–A4 公平消融给出切分/刷写效应量，并记录可复核踩坑。

关键词：持久内存；NVDIMM；文件系统；崩溃一致性；写放大；xv6；契约测试；消融实验

Abstract

Persistent memory (PMEM) offers byte-addressable access with near-DRAM latency while retaining data across power loss, which challenges the consistency model and acceptance methodology of OS storage stacks built around block devices. The teaching OS xv6 uses a virtio block device and a log-structured file system as its root file system; it is excellent for classic UNIX semantics but does not by itself provide machine-checkable crash and power-loss contracts.

This thesis implements **PMFS**, a byte-addressable persistent-memory *file subsystem* alongside the xv6 root file system: QEMU NVDIMM-window mapping, inode/dentry layout without the block I/O stack, CDC/stream chunking fused into a KV/run core, and a cache-line flush + fence persist protocol. Backends include EMUL, MAP, and file-backed PMEM2. Crash visibility is stated explicitly: only post-`fsync`/commit writes survive remount.

The distinctive contribution remains a **machine-checkable crash/power-loss contract suite** (`PERSIST`/`FILE`/`REMOUNT`/`FAULT`/`PMEM2`). Evaluation separates an architectural plane (latency/throughput magnitude versus xv6 `fs+log`) from a fairness plane (A0–A4 ablation of split/seal policy).

Keywords: persistent memory; NVDIMM; file system; crash consistency; write amplification; xv6; contract testing; ablation study

目录

摘要 **I**

Abstract **II**

第一章 绪论 **1**
1.1 研究背景 1
1.2 问题陈述 1
1.3 研究内容与目标 1
1.4 主要贡献 2
1.5 论文结构 2

第二章 相关工作 **3**
2.1 教学操作系统中的块文件系统 3
2.2 持久内存文件系统与编程模型 3
2.3 日志结构、切分与近似索引 3
2.4 作者前序原生工作与工程契约手法 3
2.5 本文定位小结 4

第三章 系统设计 **5**
3.1 产品形态与非目标 5
3.2 地址空间与三种后端 5
3.3 软件分层 5
3.4 崩溃可见点与 POSIX 差集 6
3.5 刷写协议与双槽提交 6
3.6 CDC、stream 与公平消融 7
3.7 写放大与可观测性 7
3.8 性能对比的设计意图 7

第四章 系统实现 **8**
4.1 模块与代码组织 8
4.2 双槽超级块、meta 页与 seal 8
4.3 索引路径 8
4.4 CDC / stream / 文件层 8
4.5 ONBOX_PMFS 契约套件 8
4.6 计时与评测脚手架 9
4.7 与 CI 的衔接 9

第五章 实验与分析 **10**
5.1 实验环境 10
5.2 功能、崩溃与掉电（尖刺证据） 10
5.3 性能：随 N 变化的延迟 10
5.4 写放大 12
5.5 MAP 与 EMUL 对照 12
5.6 公平消融（公平面） 12
5.7 实现难点与失败史 14
5.8 局限与威胁效度 14
5.9 本章小结 14

第六章 结论与展望 **15**
6.1 工作总结 15
6.2 不足 15
6.3 展望 15

附录 A 复现命令与扩展说明 **18**
A.1 一键契约与评测 18
A.2 关于真 NVDIMM 18
A.3 论文 PDF 编译 18

致谢 **19**

第 1 章　绪论

1.1　研究背景

操作系统存储子系统长期以块设备为中心：文件内容经缓冲缓存、日志与块 I/O 栈落盘，崩溃一致性依赖有序刷写与检查点。教学操作系统 xv6 [3] 以 virtio-blk 与日志型文件系统讲授这一经典路径，语义清晰、代码可阅读，但延迟受块设备与元数据更新约束，且难以自然表达字节寻址持久内存（Persistent Memory, PMEM）上的提交与掉电语义。

PMEM 以字节寻址提供近似 DRAM 的访问延迟，并在掉电后保留数据 [6]。学术界与工业界已出现面向 PMEM 的文件系统与编程模型, 如 PMFS [4]、NOVA [11]、BPFS [2]、Mnemosyne [10] 等。它们多面向通用或接近生产的 Linux 环境；将完整 POSIX VFS 移植进 xv6 既超出本科毕设合理体量，也易掩盖「如何验收崩溃正确性」这一更尖锐的工程问题。合题要求的「文件子系统 + 刷写/NVDIMM 仿真 + 吞吐」，需要的是**可陈述、可测的子集语义**，而非声称完整生产 FS。

1.2　问题陈述

本文关注三个相互耦合的问题:

1. **形态选择：**在 xv6 上如何提供并行于 virtio 根的 PMEM **文件子系统**（目录/extent + `create`/`fsync`），同时以 KV/run 核承载切分后的内容块，而不替换根 FS、也不假装完整 VFS?
2. **一致性模型：**双槽超级块、不可变 run、批量 seal、inode 页 magic+ 校验，如何共同保证「未 `fsync`/未提交不可见、半写活动槽与半页元数据可检测」?
3. **验收方法：**如何让崩溃、remount 字节等价与故障注入语义**可机器判定**（串口一行、宿主脚本、CI），而非仅靠人工读日志?

1.3　研究内容与目标

本文设计并实现 **PMFS**:并行于 virtio 根的 PMEM 文件子系统(`create`/`mkdir`/`fopen`/`fread`/`fwr
与融合 KV 引擎（`put`/`get`/`seal`）；写入经 CDC/stream 切分后落入同一 run 核；后端支持 EMUL、MAP 与 PMEM2。研究目标:

- 映射 QEMU NVDIMM 仿真窗；inode/dentry 去块设备布局；行级刷写协议；
- 建立含 `PERSIST`/`FILE`/`REMOUNT`/`FAULT`/`PMEM2` 的 `ONBOX_PMFS_*` 契约验收门；

- 报告文件 API 延迟中位数与相对吞吐，并以 A0–A4 公平消融给出切分/刷写效应量；
- 写死崩溃可见点与 POSIX 差集（见语义清单），使答辩口径可复核。

1.4 主要贡献

对照相关工作与作者前序原生项目（FCS / Aurora），本文贡献收敛为一条尖刺与两条支撑柱 [13, 12, 14]：

尖刺（C1）
教学 OS 上可机器判定的崩溃与掉电契约套件（含 remount 字节等价与故障注入）：相对同类，不可替代的是方法学验收，而非又一个双槽。

系统柱（C2）
去块设备文件布局 + CDC/stream 融合 KV 核 + 行级 persist + 双槽/B3/WA。

实验柱（C3）
PMEM2 掉电一行过 + A0–A4 公平消融效应量（`cycles/wa_x100`）。

主动不宣称的内容包括：完整内核 VFS、真 NVDIMM 主线化、将 FCS/Aurora 源码并入库、多写者并发、`lseek/mmap/rename`。

1.5 论文结构

第 二 章综述相关工作与借鉴边界；第 三 章给出总体设计（含可见点与差集）；第 四 章描述实现与契约；第 五 章报告实验、消融、性能图与踩坑；第 六 章总结并展望。附录给出真 NVDIMM 与复现命令。

第 2 章　相关工作

2.1　教学操作系统中的块文件系统

xv6 [3] 的 `fs+log` 路径以 inode、目录项、位图与日志检查点保证崩溃后可恢复。该设计适合教学，但其 I/O 路径绑定 virtio 块设备与 bio，元数据更新成本随「每文件一次 create/write/sync」近似线性增长。本文将该路径作为**架构面**对照基线，**仍保留为根文件系统**：PMFS 并行挂接，不主张替换日常用户程序的完整文件语义。

2.2　持久内存文件系统与编程模型

PMFS [4] 与 NOVA [11] 面向 PMEM 提供完整或接近完整的文件语义；Strata [5] 等探索跨介质分层。BPFS [2]、Mnemosyne [10] 强调细粒度持久与编程抽象。上述工作的问题域是通用或接近生产的 PMEM 文件/运行时。本文取其「字节寻址 + 显式持久」动机，但产品形态收敛为：

- **教学体量的文件子系统子集**（目录/extent + `fsync` 可见点），而非 Linux VFS 替身；
- 内容块经 CDC/stream 落入 **KV/run 核**，使刷写、索引与写放大可在同一提交路径上计量；
- 以契约套件把崩溃/掉电变成可机器判定资产，而非仅依赖手工故障演练。

相对 Linux PMFS/NOVA：差异化不在「又一个双槽/日志结构 FS」，而在 xv6 上的**方法学验收**与合题可答辩的语义边界。

2.3　日志结构、切分与近似索引

LSM 类结构 [9] 与 Bloom 过滤器 [1] 广泛用于写入友好与存在性过滤。本文 run 不可变、批量 seal 与 Bloom/fence 属于同一设计家族；CDC（内容定义切分）用于把文件写流切成可复用 chunk，定长切分仅作消融对照。实现服务于教学内核约束（页粒度、有限段数、无用户态复杂压缩管线）。

2.4　作者前序原生工作与工程契约手法

作者前序原生项目 **FCS** [13] 提供嵌入式耐久 KV 的设计约束：双槽超级块、不可变 run、B1/B2/B3 与写放大计量；**Aurora** [12] 提供键量化、二值 posting 与查询 LRU 等索引约束。二者代码不并入本仓库，xv6 侧为 C 语言自研重写，以防「移植题」误读。

工程验收上, 作者在其他内核工程中使用过 `ONBOX_*` 串口契约手法 [14]; 本课题将其重写为服务存储语义的 `ONBOX_PMFS_*` 阶段/失败码(含 `REMOUNT`/`FAULT`/`ABLATION`), 使崩溃与掉电可被 `grep` 与 CI 判定。

2.5 本文定位小结

表 2.1 相关工作与本文关系

工作	问题域	与本文
xv6 fs+log	教学块 FS	架构面对照；仍作根 FS
Linux PMFS/NOVA 等	PMEM 文件 FS	动机参考；本文为可测子集
作者 FCS / Aurora	耐久引擎与索引（原生前序）	约束下沉；代码自研重写
cxc-os 类契约	串口阶段验收	手法参考；码表重写
本文 PMFS	教学 OS 上文件子系统 + KV 核 + 契约套件	**尖刺 = 可机器判定崩溃/掉电**

防「移植题」的标准表述：数据结构思想可追溯前序约束，但验收资产（契约套件）与 xv6 内实现为本课题自有贡献；文件层与 CDC/stream 为合题后的一等公民，而非事后贴皮。

第 3 章　系统设计

3.1　产品形态与非目标

PMFS 的产品形态是：**并行于 virtio 根的字节寻址文件子系统**。用户态通过 `pmfs_create/mkdir/` 访问文件对象；并保留 `put/get/seal` 作批量引擎与融合证明。**不**替换根 FS，**不**做完整 VFS。

明确非目标：挂载替换 '/'；真物理 NVDIMM 驱动主线；并入 FCS/Aurora 源码树；`lseek/mmap/rename`/硬链接；多写者字节范围锁。

3.2　地址空间与三种后端

内核约定物理/虚拟布局（相对地址）：

Listing 3.1　PMEM 窗口相对布局示意

```
KERNBASE  ---------- 0x80000000
  kernel + user pages（易失 DRAM）
PHYSTOP   ---------- KERNBASE + 128MiB
PMEMBASE  ---------- PHYSTOP
  字节寻址持久区（双槽 SB / inode / dentry / run）
PMEMEND   ---------- PMEMBASE + 32MiB
```

表 3.1　三种物理后端

模式	构建	运行要点
EMUL	默认	$-m$ 128M，8 MiB 静态缓冲（契约 + 扫参余量）
MAP	`-DPMEM_MAP`	$-m$ 160M，映射 `[PMEMBASE, PMEMEND)`
PMEM2	`-DPMEM_MAP`	`memory-backend-file, share=on` 杀启

EMUL 用于算法与契约快速回归，**不**代表真 NVDIMM 延迟；MAP/PMEM2 验证窗口级映射与文件后端掉电存活（QEMU 文件内存后端配置见官方文档 [7]）。

3.3　软件分层

自上而下：

1. 用户 API：文件 `create/mkdir/open/read/write/fsync/readdir` 与 KV `put/get/seal`；
2. `pmfile`：目录树 + extent 元数据（inode/dentry 页，magic+ 校验）；

3. pmcdc/pmstream：Rabin 式滑动哈希切分与窗口刷写（Fixed 模式供消融）；

4. pmalgo：chunk→memtable→ 不可变 run、B1/B2/B3、融合计数 fuse_*；

5. pmindex：量化键、posting、查询 LRU（qcache）；

6. pmfs：双槽超级块与 commit；

7. pmdev/pmem_persist：行级 flush+fence，落 EMUL 或 MAP 窗口。

盘上布局：页 0/1 为超级块双槽；页 2/3 为 inode/目录项元数据；其后为 run 数据页。文件内容不以页内联为主，而以 KV chunk（extent）承载，保证与索引核同一提交路径。容量边界由实现常数约束：约 PMFILE_MAX_INO = 40、PMFILE_MAX_EXT = 4、PM_VAL_MAX = 512，单文件量级约 2 KiB（刻意不声称大文件生产能力）。

3.4 崩溃可见点与 POSIX 差集

设计层先写死「哪些写在 remount/掉电后可见」，再谈实现：

表 3.2 崩溃一致可见点（设计契约）

操作	remount 后可见?	机制要点
write（未 fsync）	**否**	chunk 可在 memtable/stream；inode extent 未落页
fsync/fclose(dirty)	**是**	stream flush→pm_seal + meta 页 + SB commit
KV put 未 seal	**否**	与文件 nosync 同模型
活动 SB / inode 半页撕坏	检测/拒绝	回退另一槽；magic+chk 拒绝垃圾真相

与 POSIX 的差集（主动不实现）包括：多层完整 namei、O_APPEND/lseek、多进程共享 fd、rename/硬链接、mmap、权限模型、根挂载替换。用户态 pmfs_posix 走真实 mkdir/readdir/fsync/reopen，用于证明文件 API 深化，而非仅 path-hash 到 KV 的薄 shim。

3.5 刷写协议与双槽提交

持久化协议固定为：

$$\text{store} \;\rightarrow\; \text{flush}(64\text{B lines}) \;\rightarrow\; \text{fence} \;\rightarrow\; \text{store SB} \;\rightarrow\; \text{flush} \;\rightarrow\; \text{fence}.$$

文件路径上，fsync = 刷 stream 窗 + 写 inode/dentry 页 + pmfs_commit。提交以 seal/commit 为显式边界：批量 put 进入内存表后，seal 将 run 持久化并原子翻转活动超级块槽位

（epoch + 校验）。设计保证：半写活动槽可检测并回退；未提交数据对读路径不可见；索引可自 run 重建。

3.6 CDC、stream 与公平消融

文件写流经 Rabin 式 CDC 切分为 chunk（定长 Fixed 仅作对照），再经 stream 窗口聚合后 `pm_put`；flush/seal 边界由策略开关控制。公平消融 A0–A4 在**同一 payload、同一页设备与 KV 核**上只切换「CDC 开/关」与「批量 seal / 每 chunk 立即 seal」，输出 `cycles` 与 `wa_x100` 作为公平面效应量。架构面（相对 xv6 日志栈的量级差）与公平面（策略效应）**分表陈述、禁止混读**。

3.7 写放大与可观测性

定义

$$\text{wa_x100} = \frac{\text{phys} \times 100}{\text{logical}},$$

其中 logical 为用户 payload 字节，phys 为整页写入（对齐、run、B3 重写）。小 value（如 8 B）相对 4 KiB 页必然放大；该放大与 xv6 日志放大**不同源**，论文中分开叙述。`fuse_puts`/`fuse_seals` 用于证明文件路径未旁路 KV 核。

3.8 性能对比的设计意图

实验对比三条路径：PMFS-KV、PMFS-File、xv6 `fs+log`。File 路径为 N 次小文件 `create/write/fsync`；KV 为 N 次 `put` + 一次 `seal` + N 次 `get`；xv6 为每键一次小文件 + `sync`。目的是说明「避免 virtio 日志与块栈」的**量级**差异，而非宣称同等完整 POSIX 下的绝对加速比。公平面效应量只引用 A0–A4。

第 4 章　系统实现

4.1　模块与代码组织

实现基于 xv6-riscv，主要模块包括：pmem/pmfs（后端与双槽）、pmalgo/pmindex（KV 与索引核）、pmcdc/pmstream（切分与滑窗）、pmfile/syspmfile（目录 +extent 文件层）、pmcontract（串口契约），以及用户态 _pmfs_probe/_pmfs_file/_pmfs_bench/_pmfs_posix 文件写路径强制经 pm_put/pm_seal，由 fuse_puts/fuse_seals 计数证明未旁路。

4.2　双槽超级块、meta 页与 seal

活动槽与镜像槽交替提交。seal 路径将当前 mem 表固化为 run 页，更新超级块字段（epoch、校验、run 元数据），再经 pmem_persist 落后端。挂载时选择校验合法且 epoch 较新的槽；半写损坏触发 CRASH 相关失败码并回退。

inode/dentry 页带 magic 与校验和：加载失败则拒绝把损坏页当作真相（契约 FAULT_HALFPAGE）。B3 路径对需重写的页做真页写入，而非仅更新指针；pmfs_stat 导出 logical/phys 与 wa_x100。段数上限（PM_MAX_RUNS 等）在扫参与多 trial 下曾成为楔死点，实现侧提高余量并在 attach 压力下淘汰最旧段视图，保证契约与 bench 可重复跑通。

4.3　索引路径

索引采用键量化与 posting 结构，查询侧维护 LRU 缓存（qcache）。pm_get 的 hint 仅为软提示：命中失败时回退完整查找，并在故障注入后使脏 dent 失效，避免「缓存撒谎」。索引可自 run 重建：在挂载或自测阶段扫描已提交 run，避免「索引已更新而数据未提交」的窗口。该约束来自 Aurora 前序设计 [12]，实现为 xv6 C 重写。

4.4　CDC / stream / 文件层

pmcdc 实现 Rabin 滑动哈希与 Fixed 定长两种切分；pmstream 维护写窗并在策略要求时 seal。pmfile 将 chunk 记为 extent，读路径按 extent 拼装；若读时空内容而 stream 已有数据，则从 stream 同步 extent，避免「写成功但读空」的假失败。目录项保存路径与 parent，支持有限深度的 mkdir/readdir，非完整 VFS namei。

4.5　ONBOX_PMFS 契约套件

所有契约行必须可被 grep / tools/contract_pmfs.py 匹配。

成功行

ONBOX_PMFS_⟨STAGE⟩_PASS + 空格分隔 key=value

失败行

ONBOX_PMFS_⟨STAGE⟩_FAIL stage=... code=...

总闸

必过子集全部成功后打印 ONBOX_PMFS_PASS

必过子集（平级）：PMEM0、MOUNT、STORE、INDEX、ALGO、CRASH、PERSIST、FILE、CDC、STREAM、POS_FILE、NEG_FILE、CHAOS_FILE、ABLATION、REMOUNT、FAULT，然后总闸。make contract-strict 另检 POS/NEG CDC/STREAM、EDGE_FILE、A0–A4（含 wa_x100=）、FAULT_SEALMID/FAULT_HALFPAGE。

语义证明行（尖刺证据的核心子集）：

```
ONBOX_PMFS_REMOUNT_PASS bytes=... equal=1
ONBOX_PMFS_FAULT_SEALMID_PASS invisible=1
ONBOX_PMFS_FAULT_HALFPAGE_PASS chk_reject=1
ONBOX_PMFS_PMEM2_PASS persist=ok
```

MAP 模式额外要求 PMEM0 mode=MAP 与 PMEM1。PMEM2 脚本成功时再印 ONBOX_PMFS_PMEM2_CO
boots=2。

用户态：_pmfs_probe → SYSCALL_PASS；_pmfs_file → FILEUSER_PASS；
_pmfs_bench [N] → BENCH_PASS；_pmfs_posix → POSIX_PASS（真实 mkdir/readdir/fsync
不升主线为根 FS）。

4.6 计时与评测脚手架

用户态原调用 cycles() 依赖 rdcycle；在未开放 mcounteren.CY 时会触发非法指令（计数器与特权态约束见 RISC-V 特权规范 [8]）。实现将 sys_cycles 改为已允许的 r_time()，文档明确其为**相对时间单位**，不作墙钟微秒解读。评测脚本对每个 N 执行 5 次 trial 取中位数，**每 N 独立开机**，并等待契约行字段齐全后再杀 QEMU，避免截断解析与段数楔死。

4.7 与 CI 的衔接

持续集成挂接 usertests 与契约目标（含 strict），使回归不以人工读完整串口日志为前提。尖刺价值正在于此：正确性与掉电语义「grep 一行即可判过」。

第 5 章　实验与分析

5.1　实验环境

表 5.1　实验环境

项	配置
基线	xv6-riscv（课题分支）
模拟器	QEMU `virt`，`CPUS=1`
后端	EMUL（8 MiB）/ MAP / PMEM2（见第 三 章）
回归	`usertests`、`contract`/`contract-strict`/MAP/PMEM2（CI）
计时	内核 `r_time()` 相对单位；每 N 取 5 trial 中位数；每 N 独立开机

数据复现：`make bench-sweep` 回填 `docs/BENCH.md`；契约：`make contract`、`contract-strict`、`contract-pmem`、`contract-pmem2`。公平消融见 `docs/ABLATION.md`。语义与故障矩阵见第 3.4 节与 `docs/SEMANTICS.md`。

两套结论：架构面——相对 xv6 日志路径的量级差；公平面——同载荷消融效应量。禁止把 `fratio` 读成同等 POSIX API 加速比。

5.2　功能、崩溃与掉电（尖刺证据）

表 5.2　契约验收用例

用例	命令/阶段	期望	说明
总闸	`make contract`	`ONBOX_PMFS_PASS`	EMUL 必过子集
严格	`contract-strict`	同上 + A0–A4/`FAULT_*`	POS/NEG/效应量
MAP	`contract-pmem`	`mode=MAP`+PMEM1	窗口映射
掉电	`contract-pmem2`	`PMEM2_PASS`	文件后端杀启
remount	`REMOUNT`	`equal=1`	sync 后字节等价
故障	`FAULT_*`	不可见 / chk 拒绝	未 sync；半页撕坏
POSIX	`pmfs_posix`	`POSIX_PASS`	真实文件 API，不升主线

分析：契约套件使「完好 SB / 未提交不可见 / remount 等价 / 半页拒绝 / 窗口 persist」可机器判定；相对仅演示双槽结构，这是方法学差异，构成本文尖刺。

5.3　性能：随 N 变化的延迟

对比路径故意不对等（表 5.3）。ratio$\gg 1$ 主要反映避免 virtio 日志与块栈的架构差，**不可**解读为同等完整 POSIX 下的千倍加速。

表 5.3　故意不对等的对比维度

维度	PMFS-KV / File	xv6 fs+log
API	KV：put×N+seal+get；File：create/write/fsync×N	open/create+write(8B)×N+sync
持久语义	双槽 SB + inode/dentry 页	块日志检查点
I/O 栈	字节寻址窗口 / EMUL	virtio-blk + bio + log
元数据	有限 inode/dentry（无块位图日志）	目录 +inode+ 位图 +log

表 5.4　EMUL 下随 N 变化的中位数与相对吞吐（5 trial，CDC 融合后）

N	med_kv	med_file	med_xv6	fratio×100	thr_file	wa_x100
8	33507	304451	5684939	1867	26	15384
16	38448	486211	11208338	2305	32	17889
32	53065	659056	25648605	3891	48	17721

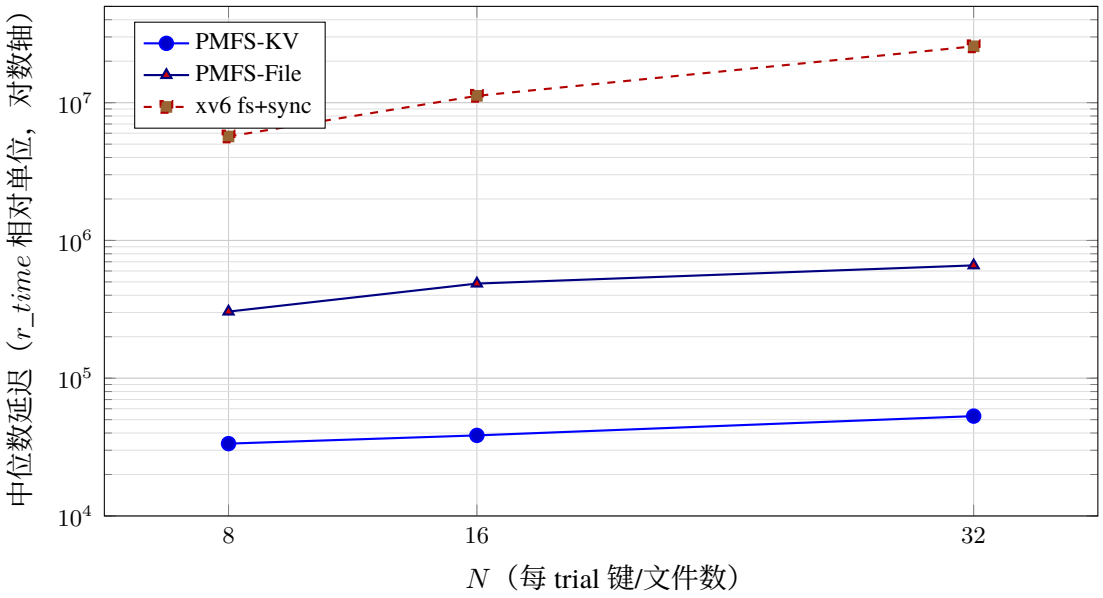


图 5.1　随 N 变化的中位数延迟（EMUL，5 trial，CDC 融合后）。对数轴强调相对 xv6 的量级差，不作同等 POSIX 加速比解读。

观察：（1）xv6 随 N 近似线性；（2）PMFS-File 每文件 fsync 仍避开块日志栈，fratio$\gg 1$；（3）File 路径因 CDC/stream/meta 提交高于纯 KV，但仍与 xv6 差一个数量级以上；（4）结论写作**量级优势 + 边界**。

5.4 写放大

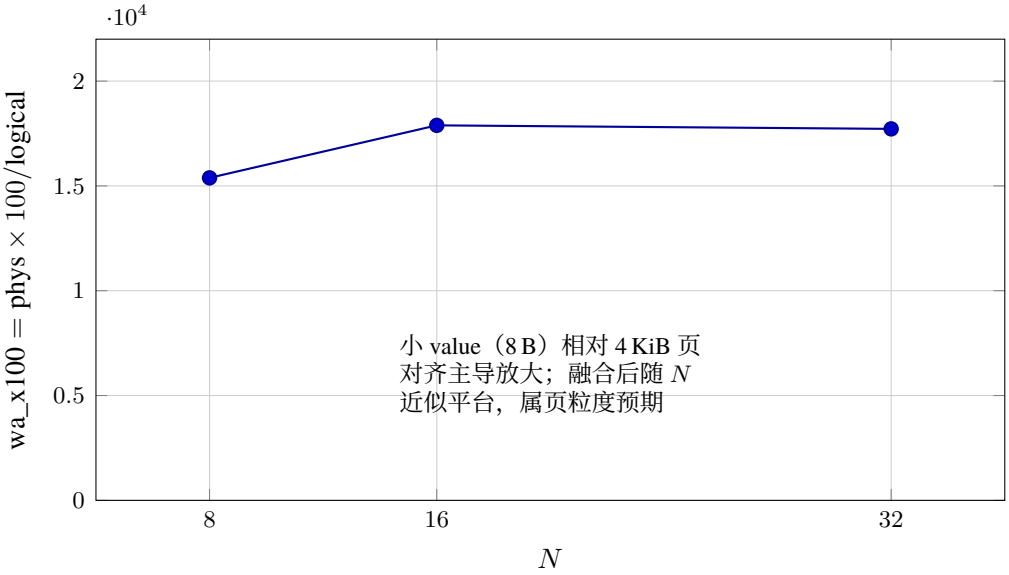


图 5.2　写放大 wa_x100 随 N 变化（EMUL，KV 路径）。高放大属页粒度预期，与 xv6 日志放大不同源。

小 value（8 B）相对 4 KiB 页对齐主导放大。CDC 融合后本负载下 wa_x100 维持在高位且随 N 近似平台（表 5.4），属页粒度与 run 打包的预期行为；与 xv6 日志放大不同源，不可横向比绝对值。

5.5 MAP 与 EMUL 对照

表 5.5　N=32 时 EMUL 与 MAP

mode	med_kv	med_file	med_xv6	fratio×100
EMUL	53065	659056	25648605	3891
MAP	113804	4301198	24439543	568

文件路径相对吞吐在 EMUL 下稳定高于 xv6 一个数量级以上（表中 `thr_file`）。MAP 抬升软件路径开销后 fratio 下降但仍 $\gg 1$。EMUL 仍**不**代表真 NVDIMM 延迟。

5.6 公平消融（公平面）

同 payload 下，关闭批量 seal（A2/A3）显著抬升 wa_x100 与 cycles；A0/A4 同配置复测同量级，说明效应量可重复。公平面只据此表陈述，**不**与表 5.4 的 fratio 混读。



图 5.3　N=32 时 PMFS-KV 在 EMUL 与 MAP 下的中位数延迟（CDC 融合后）。

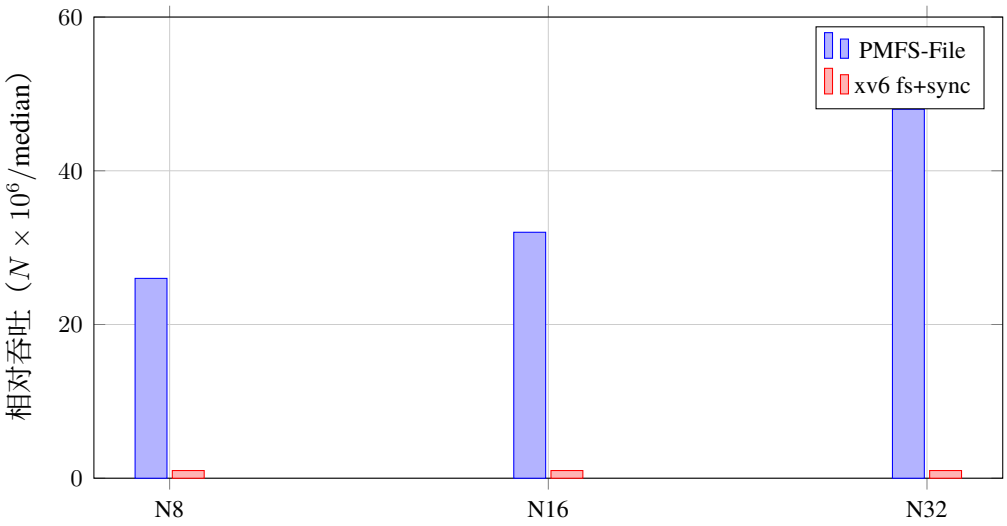


图 5.4　EMUL 下文件 API 相对吞吐（合题「吞吐」指标；非墙钟 MB/s）。

表 5.6　A0–A4 公平消融效应量（EMUL，`contract-strict` 实测）

变体	CDC	batch seal	cycles	wa_x100	解读
A0	开	开	30029	1600	默认策略
A1	关	开	27490	1600	定长 + 批量
A2	开	关	52066	4800	每 chunk seal，WA/时延升
A3	关	关	36238	3200	定长 + 立即 seal
A4	开	开	24146	1600	A0 复测（稳定性）

5.7 实现难点与失败史

下列踩坑用于证明系统是「调出来的」，并可复核。

P0 启动栈溢出 `main()` 跑在原 4 KiB/hart 的 `stack0` 上；自测栈上放置多页缓冲导致溢出，污染邻近内存。修复：页缓冲改 `static`；`stack0` 扩至 16 KiB/hart。

P1 `rdcycle` 非法指令 `timerinit` 未开放 `CY` 时，S 态 `rdcycle` 触发 `kerneltrap`。修复：改用已允许的 `r_time()`，并在论文中不作墙钟 μs 解读。

P2 QEMU NUMA / RAM 尺寸不匹配 仅配置 160 M 文件后端而默认 $-m$ 仍为 128 M 时，QEMU 立即退出，契约表现为超时。修复：文件后端启动同步 $-m$ 160M。

P3 MAP 在 $-m$ 128M 下写 PMEMBASE 无物理后备窗口上的写探测导致 store fault。修复：仅 `-DPMEM_MAP` 时映射并探测。

P4 Bench 过早杀进程截断契约行 匹配到 `BENCH_PASS` 子串即杀 QEMU，导致字段截断。修复：等待 `median_xv6=` / `phys=` / `wa_x100=` 齐全。

P5 多 trial 撑爆段数 连续 seal 耗尽段视图导致后续 trial 失败。修复：提高 run/段余量、attach 压力阀、**每 N 独立开机**扫参。

P6 题面「文件系统」与实现漂移 早期曾偏并行 KV。修复：升格 dir/extent/CDC + 深化 `pmfs_posix`，并以语义清单写死差集，避免题实不符。

5.8 局限与威胁效度

1. QEMU 非真 NVDIMM；附录给出扩展路径但不喧宾夺主。
2. 架构面性能对比 API 异构；结论限于量级说明。公平面仅 A0–A4 同载荷可比。
3. 文件语义为可测子集：无 `lseek`/`mmap`/`rename`；容量受 extent/chunk 上限约束；默认 `CPUS=1`。
4. 计时为相对单位；外推墙钟或真机延迟需重新校准。

5.9 本章小结

尖刺（契约套件，含 remount/故障）+ 支撑柱（文件子系统/KV 核/PMEM2）+ 两套结论口径 + 可复核踩坑，构成实验章叙事主线；性能图服务于量级与写放大机制说明，消融表服务于公平面效应量，而非绝对公平微基准夺冠。

第 6 章　结论与展望

6.1　工作总结

本文在 xv6-riscv 上设计并实现了并行于根文件系统的字节寻址 **PMFS 文件子系统**：目录与 extent 元数据、CDC/stream 切分写入、融合至 KV run/索引核，并覆盖 EMUL / MAP / PMEM2 三种后端。系统以双槽提交、行级 flush+fence、inode 页 magic+ 校验，以及 `ONBOX_PMFS_*` 契约套件（含 remount 字节等价与故障注入）验收崩溃与掉电语义。

主要结论如下：

1. **方法学：**相对「再实现一套双槽 PMEM-FS」，教学 OS 上可机器判定的契约套件（含 `REMOUNT`/`FAULT`/`PMEM2`）才是不可替代点；正确性可用一行串口与 CI 复现。
2. **系统：**明确崩溃可见点——仅 `fsync`/提交后写可见；未 sync 写在 remount 后不可见；半页元数据损坏被校验拒绝。文件路径经 CDC/stream 融合 KV 核，`fuse_*` 可证未旁路。
3. **实验：**架构面报告相对 xv6 `fs+log` 的量级延迟/吞吐差；公平面以 A0–A4 同载荷消融给出 `cycles`/`wa_x100` 效应量（关闭批量 seal 抬升 WA），二者不可混读。

6.2　不足

真物理 NVDIMM、完整 POSIX（seek/mmap/rename/多写者）、多核并发均未纳入主线。性能数字依赖相对时间单位与 QEMU，外推需谨慎。文件容量受 extent 槽与 chunk 上限约束（约 2 KiB/文件量级）。

6.3　展望

- 在保持契约码表稳定的前提下扩展更多页损坏与乱序代理模式；
- 在附录路线指引下对接真实 PMEM/NVDIMM 做延迟校准；
- 在不破坏尖刺的前提下增强大文件与只读共享语义的可测边界。

答辩一句话总结：**不可替代点是教学 OS 上可机器判定的崩溃/掉电契约套件（含 remount 内容等价）；量级优势来自去块设备路径的架构差，公平面消融另表陈述。**

参考文献

[1] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[2] Jeremy Condit, Edmund B. Nightingale, Christopher Frost, Engin Ipek, Benjamin C. Lee, Doug Burger, and Derrick Coetzee. Better I/O through byte-addressable, persistent memory. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pages 133–146. ACM, 2009. 提出 BPFS.

[3] Russ Cox, M. Frans Kaashoek, and Robert Morris. *xv6: a simple, Unix-like teaching operating system*. MIT CSAIL, 2022. RISC-V edition; https://pdos.csail.mit.edu/6.1810/.

[4] Subramanya R. Dulloor, Sanjay Kumar, Anil Keshavamurthy, Philip Lantz, Dheeraj Reddy, Rajesh Sankaran, and Jeff Jackson. System software for persistent memory. In *Proceedings of the Ninth European Conference on Computer Systems (EuroSys)*, pages 1–15. ACM, 2014. 提出并实现 PMFS；EuroSys 论文题名为 System Software for Persistent Memory.

[5] Youngjin Kwon, Henrique Fingler, Tyler Hunt, Simon Peter, Emmett Witchel, and Thomas Anderson. Strata: A cross media file system. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 460–477. ACM, 2017.

[6] Hai-Kun Liu, Di Chen, Hai Jin, Xiao-Fei Liao, Binsheng He, Kan Hu, and Yu Zhang. A survey of non-volatile main memory technologies: State-of-the-arts, practices, and future directions. *Journal of Computer Science and Technology*, 36(1):4–32, 2021.

[7] QEMU Project. *QEMU Documentation*, 2024. https://www.qemu.org/docs/master/.

[8] RISC-V International. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, 2021. Privileged Spec; https://riscv.org/technical/specifications/.

[9] Russell Sears and Raghu Ramakrishnan. bLSM: A general purpose log structured merge tree. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 217–228. ACM, 2012.

[10] Haris Volos, Andres Jaan Tack, and Michael M. Swift. Mnemosyne: Lightweight persistent memory. In *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 91–104. ACM, 2011.

[11] Jian Xu and Steven Swanson. NOVA: A log-structured file system for hybrid Volatile/Non-volatile main memories. In *14th USENIX Conference on File and Storage Technologies (FAST)*, pages 323–338, Santa Clara, CA, 2016. USENIX Association.

[12] 作者. Aurora：量化索引与查询缓存（作者前序原生项目）. 未公开发表设计文档与实现, 2024. 设计约束：键量化、二值 posting、查询 LRU；代码不并入本仓库.

[13] 作者. FCS：嵌入式耐久键值存储引擎（作者前序原生项目）. 未公开发表设计文档与实现, 2024. 设计约束：双槽超级块、不可变 run、B1/B2/B3、写放大计量；代码不并入本仓库.

[14] 作者. CXC/OS 工程化契约验收手法. 作者相关工程实践, 2025. ONBOX 串口契约思想；本课题重写为 ONBOX_PMFS_*.

第 A 章　复现命令与扩展说明

A.1　一键契约与评测

在仓库根目录（WSL）执行：

```
make contract            # EMUL 总闸 -> ONBOX_PMFS_PASS
make contract-strict     # 另检 POS/NEG、A0--A4、FAULT_*
make contract-pmem       # MAP
make contract-pmem2      # 文件后端掉电二次启动
make bench-sweep         # N=8/16/32 中位数，回填 docs/BENCH.md
```

宿主解析脚本位于 `tools/contract_pmfs.py`、`tools/contract_pmem2.py` 与 bench 相关脚本。语义清单与故障矩阵见仓库 `docs/SEMANTICS.md`；消融口径见 `docs/ABLATION.md`。

A.2　关于真 NVDIMM

主线以 QEMU 窗口与文件后端验证语义与契约。若需真 NVDIMM：在具备硬件与内核/固件支持的平台上，将 MAP 窗口替换为实际持久区映射，并重新校准延迟与 `pmem_persist` 屏障语义。该路径不改变契约阶段码表的方法学，但会影响绝对延迟数字；故正文不以其为依赖。

A.3　论文 PDF 编译

```
cd thesis/latex
make          # xelatex + bibtex + xelatex x2
```

要求：TeX Live（含 `ctex`、`pgfplots`）、Noto CJK 字体。封面姓名/学院等宏可在 `main.tex` 中替换为学校正式模板。

致　谢

感谢指导教师在选题、系统边界与论文表述上的悉心指导。感谢教学操作系统 xv6 与 RISC-V / QEMU 开源社区提供的可读代码与实验平台。感谢课程与实验室同学在讨论、复现与答辩准备中的帮助。

本课题将工程验收（可机器判定契约）置于与数据结构同等重要的位置；文中踩坑记录旨在便于后来者复核。文责自负，疏漏之处恳请批评指正。